

به نام خدا

عنوان پروژه: طراحی و پیاده سازی پایگاه داده سیستم انتخاب واحد دانشگاهی

استاد راهنما: جناب آقای دکتر علی طائی زاده

دانشجویان: سعیده سادات هاشمی نژاد

تاریخ تحویل: ۶ اسفند ۱۴۰۴

۱. مقدمه

در این پروژه، یک سیستم پایگاه داده برای مدیریت انتخاب واحد دانشگاهی طراحی و پیاده‌سازی شده است. هدف از این پروژه، مدل‌سازی فرآیند انتخاب واحد به گونه‌ای است که بتوان اطلاعات دانشجویان، اساتید، دروس، کلاس‌ها و ثبت‌نام‌ها را به صورت ساختاریافته و بهینه ذخیره و مدیریت کرد. این سیستم بخشی از یک سیستم جامع‌تر دانشگاهی است و با تمرکز بر زیرسیستم انتخاب واحد طراحی شده است. پیاده‌سازی این پروژه در محیط Navicat انجام شده و داده‌های آزمایشی واقع‌گرایانه‌ای برای آزمون صحت عملکرد جداول و کوئری‌ها درج شده‌اند.

۲. تحلیل نیازمندی‌ها

سیستم انتخاب واحد دانشگاهی باید قابلیت‌های زیر را پوشش دهد:

- نگهداری اطلاعات کامل دانشجویان شامل مشخصات شخصی و رشته تحصیلی.
- مدیریت ساختار سازمانی دانشگاه شامل دانشکده‌ها و رشته‌ها.
- ذخیره‌سازی اطلاعات دروس و پیش‌نیازهای آن‌ها.
- مدیریت اطلاعات اساتید و دانشکده‌ی مربوط به هر استاد.
- برنامه‌ریزی کلاس‌ها بر اساس ترم تحصیلی.
- استاد و ظرفیت.
- ثبت و مدیریت فرآیند انتخاب واحد دانشجویان به همراه نمرات و وضعیت ثبت‌نام.

۳. نمودار ER

نمودار موجودیت-ارتباط (Entity-Relationship) این سیستم بر اساس تحلیل نیازمندی‌ها طراحی شده است. موجودیت‌های اصلی شناسایی شده در این سیستم عبارت‌اند از: دانشجو، رشته، دانشکده، درس، پیش‌نیاز، کلاس، ترم، استاد و ثبت‌نام. روابط میان این موجودیت‌ها به گونه‌ای تعریف شده‌اند که یکپارچگی داده‌ها تضمین شود. برای نمونه، هر دانشجو به یک رشته تعلق دارد، هر رشته به یک دانشکده وابسته است، هر کلاس توسط یک استاد در یک ترم مشخص ارائه می‌شود و هر ثبت‌نام دانشجویی دقیقاً به یک کلاس اشاره دارد.

۴. طراحی پایگاه داده (ساختار جداول)

پایگاه داده این سیستم از ۹ جدول تشکیل شده است که در ادامه هر یک توضیح داده می‌شود.

۱) جدول students (دانشجویان)

این جدول اطلاعات شخصی و تحصیلی دانشجویان را نگهداری می‌کند.

Students	
StudentID	int
FirstName	VARCHAR(100)
LastName	VARCHAR(100)
DateOfBirth	DATE
MajorID	int
Gender	ENUM(Male, Female)
Email	Email
Phone	VARCHAR(15)

فیلد MajorID به عنوان کلید خارجی به جدول Major متصل است و رشته تحصیلی هر دانشجو را مشخص می‌کند.

۲) جدول Major (رشته‌ی تحصیلی)

این جدول رشته‌های تحصیلی موجود در دانشگاه را تعریف می‌کند و هر رشته را به دانشکده مربوط خود متصل می‌نماید.

Major	
MajorID 	integer
MajorName	VARCHAR(100)
DepartmentID	int

۳) جدول Department (دانشکده)

این جدول اطلاعات دانشکده‌های دانشگاه را ذخیره می‌کند.

Department 	
DepartmentID 	INT
DepartmentName	VARCHAR(100)
Location	VARCHAR(100)

۴) جدول Courses (دروس)

این جدول اطلاعات کلیه دروس ارائه شده در دانشگاه را نگهداری می کند.

Courses	
CourseID 	INT
CourseName	VARCHAR(100)
Credits	INT
Description	VARCHAR(500)
DepartmentID	INT

۵) جدول Prerequisite (پیش نیاز)

این جدول یک جدول ارتباطی مستقل است که رابطه پیش نیازی میان دروس را مدل می کند. ساختار

این جدول اجازه می دهد یک درس چندین پیش نیاز داشته باشد.

Prerequisite 	
CourseID 	int
PrerequisiteID 	int

هر دو فیلد این جدول به کلید اصلی CourseID در جدول Courses اشاره می کنند و با هم کلید اصلی

ترکیبی این جدول را تشکیل می دهند.

۶) جدول Semester (ترم تحصیلی)

این جدول ترم‌های تحصیلی دانشگاه را ذخیره می‌کند. داده‌های این جدول از سال ۱۴۰۰ تا ۱۴۰۴ شمسی را پوشش می‌دهند و به ازای هر سال تحصیلی دو ترم در نظر گرفته شده است. بنابراین مقادیر فیلد TermInfo به صورت ۱، ۲، ۳، ۴، ۵، ۶ و ۷ به همین ترتیب تا ۱۴۰۴ تعریف می‌شوند.

Semester	
SemesterID 	INT
TermInfo	VARCHAR(5)

۷) جدول Instructor (اساتید)

این جدول اطلاعات اساتید دانشگاه را نگهداری می‌کند.

Instructor	
InstructorID 	INT
FirstName	VARCHAR(100)
LastName	VARCHAR(100)
Email	Email
PhoneNumber	VARCHAR(15)
DepartmentID	int

فیلد DepartmentID مشخص می‌کند که هر استاد در کدام دانشکده تدریس می‌کند.

۸) جدول Section (کلاس‌ها)

این جدول اطلاعات مربوط به برگزاری هر کلاس درسی را ذخیره می‌کند. یک درس می‌تواند در یک ترم چندین section (گروه) داشته باشد.

Section	
SectionID 🔑	INT
SectionNumber	int
CourseID	int
InstructorID	int
SemesterID	int
Capacity	int
TimeSlot	datetime
Location	VARCHAR(100)

۹) جدول Enrollment (انتخاب واحد)

این جدول، هسته اصلی سیستم انتخاب واحد است و ارتباط میان دانشجو و کلاس انتخاب‌شده را ثبت می‌کند.

Enrollment	
EnrollmentID 	integer
StudentID	int
SectionID	int
Grade	int
EnrollmentDate	date
Status	ENUM(Registered, Dropped)

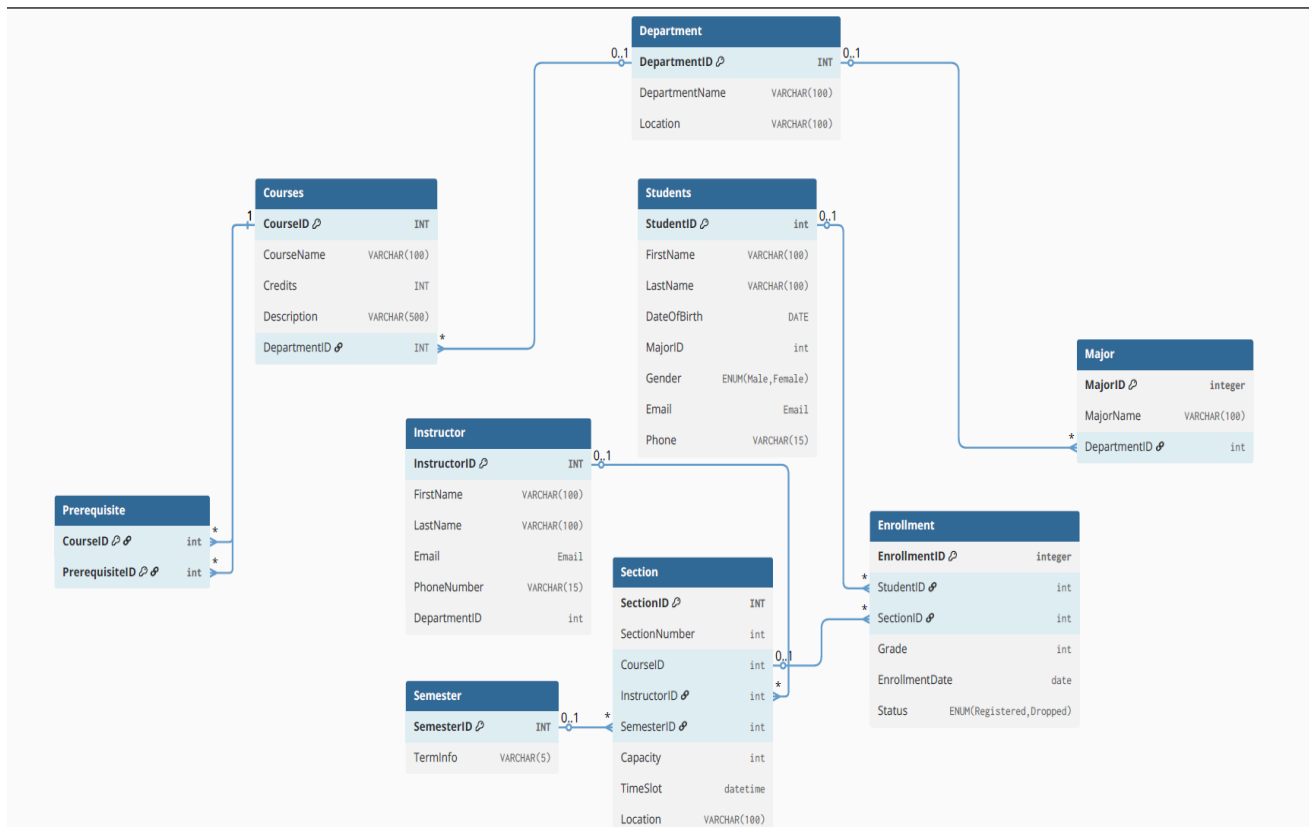
فیلد Status نشان می‌دهد که آیا دانشجو هنوز در درس ثبت‌نام دارد یا آن را حذف کرده است.

۵. روابط میان جداول

روابط تعریف‌شده در این پایگاه داده به صورت زیر است:

- هر دانشجو (Students) به یک رشته (Major) تعلق دارد — رابطه چند به یک.
- هر رشته (Major) به یک دانشکده (Department) تعلق دارد — رابطه چند به یک.
- هر درس (Courses) به یک دانشکده (Department) تعلق دارد — رابطه چند به یک.
- هر استاد (Instructor) در یک دانشکده (Department) تدریس می‌کند — رابطه چند به یک.
- هر کلاس (Section) به یک درس، یک استاد و یک ترم وابسته است — روابط چند به یک.
- جدول Prerequisite رابطه خودارجاعی روی جدول Courses پیاده‌سازی می‌کند.
- هر ثبت‌نام (Enrollment) یک دانشجو را به یک کلاس متصل می‌کند — رابطه چند به چند که از طریق این جدول میانی حل شده است.

۶. نمای کلی جداول



۷. کوئری‌های SQL

در این بخش، کوئری‌های نوشته‌شده برای سیستم به سه دسته تقسیم شده‌اند: کوئری‌های ساده، کوئری‌های با

JOIN و کوئری‌های با GROUP BY و توابع تجمیعی.

۱) کوئری‌های ساده (Simple Queries)

▪ کوئری ۱ - نمایش تمام دانشجویان:

```
SELECT * FROM Students;
```

این کوئری تمام رکوردها و فیلدهای جدول دانشجویان را بازمی گرداند و برای مرور کلی داده‌ها استفاده می‌شود.

▪ کوئری ۲- نمایش نام و ایمیل دانشجویان:

```
SELECT FirstName, LastName, Email FROM Students;
```

در این کوئری با انتخاب ستون‌های مشخص، فقط اطلاعات ضروری نمایش داده می‌شود که باعث خوانایی بهتر و کاهش حجم داده انتقالی می‌شود.

▪ کوئری ۳- نمایش دانشجویان خانم:

```
SELECT FirstName, LastName FROM Students
```

```
WHERE Gender = 'Female';
```

با استفاده از شرط WHERE، دانشجویانی که جنسیت آن‌ها Female است فیلتر می‌شوند.

▪ کوئری ۴- نمایش دروس بیش از ۲ واحد:

```
SELECT CourseName, Credits FROM Courses
```

```
WHERE Credits > 2;
```

این کوئری دروسی را که واحد آن‌ها بیشتر از ۲ است نمایش می‌دهد.

▪ کوئری ۵- نمایش ثبت‌نام‌های فعال:

```
SELECT * FROM Enrollment
```

```
WHERE Status = 'Registered';
```

رکوردهایی از جدول Enrollment که وضعیت آنها Registered است نمایش داده می‌شوند، یعنی دانشجویانی که درس را حذف نکرده‌اند.

۲) کوئری‌های با JOIN

▪ کوئری ۶- نمایش نام دانشجو به همراه رشته تحصیلی:

```
SELECT s.FirstName, s.LastName, m.MajorName  
FROM Students s  
JOIN Major m ON s.MajorID = m.MajorID;
```

با استفاده از INNER JOIN میان جداول Students و Major، نام هر دانشجو در کنار رشته تحصیلی‌اش نمایش داده می‌شود.

▪ کوئری ۷- نمایش نام دانشجو به همراه دانشکده:

```
SELECT s.FirstName, s.LastName, d.DepartmentName  
FROM Students s  
JOIN Major m ON s.MajorID = m.MajorID  
JOIN Department d ON m.DepartmentID = d.DepartmentID;
```

در این کوئری با دو JOIN متوالی، مسیر $\text{Students} \leftarrow \text{Major} \leftarrow \text{Department}$ طی می‌شود تا دانشکده هر دانشجو مشخص شود.

▪ کوئری ۸- نمایش دروس انتخابی هر دانشجو:

```
SELECT s.FirstName, s.LastName, c.CourseName, e.Status  
FROM Enrollment e
```

JOIN Students s ON e.StudentID = s.StudentID

JOIN Section sec ON e.SectionID = sec.SectionID

JOIN Courses c ON sec.CourseID = c.CourseID;

این کوئری یکی از مهم‌ترین کوئری‌های سیستم است. با پیوند چهار جدول، مشخص می‌شود که هر دانشجو در کدام دروس ثبت‌نام کرده است.

▪ کوئری ۹- نمایش کلاس‌ها به همراه استاد و ترم:

SELECT c.CourseName, i.FirstName, i.LastName, sem.TermInfo, sec.Location

FROM Section sec

JOIN Courses c ON sec.CourseID = c.CourseID

JOIN Instructor i ON sec.InstructorID = i.InstructorID

JOIN Semester sem ON sec.SemesterID = sem.SemesterID;

این کوئری با پیوند چهار جدول، جدول زمانی کلاس‌ها را به همراه نام استاد، نام درس، ترم و مکان برگزاری نمایش می‌دهد.

▪ کوئری ۱۰- نمایش پیش‌نیازهای هر درس:

SELECT c1.CourseName AS 'درس اصلی', c2.CourseName AS 'پیش‌نیاز'

FROM Prerequisite p

JOIN Courses c1 ON p.CourseID = c1.CourseID

JOIN Courses c2 ON p.PrerequisiteID = c2.CourseID;

از آنجا که جدول Prerequisite دو کلید خارجی به جدول Courses دارد، در این کوئری جدول Courses دوبار با نام مستعار متفاوت JOIN شده تا هم نام درس اصلی و هم نام درس پیش‌نیاز نمایش داده شود.

۳) کوئری های تجمیعی

▪ کوئری ۱۱- تعداد دانشجویان هر رشته:

```
SELECT m.MajorName, COUNT(s.StudentID) AS تعداد_دانشجو
FROM Students s
JOIN Major m ON s.MajorID = m.MajorID
GROUP BY m.MajorName;
```

با استفاده از تابع COUNT و GROUP BY ، تعداد دانشجویان در هر رشته محاسبه می‌شود.

▪ کوئری ۱۲- میانگین نمره در هر درس:

```
SELECT c.CourseName, AVG(e.Grade) AS میانگین_نمره
FROM Enrollment e
JOIN Section sec ON e.SectionID = sec.SectionID
JOIN Courses c ON sec.CourseID = c.CourseID
GROUP BY c.CourseName;
```

تابع AVG میانگین نمرات دانشجویان را به ازای هر درس محاسبه می‌کند. این اطلاعات

می‌تواند برای ارزیابی سطح دشواری دروس مفید باشد.

- کوئری ۱۳ - تعداد کلاس‌های هر استاد:

```
SELECT i.FirstName, i.LastName, COUNT(sec.SectionID) AS تعداد_کلاس  
FROM Instructor i  
JOIN Section sec ON i.InstructorID = sec.InstructorID  
GROUP BY i.InstructorID;
```

تعداد کلاس‌هایی که هر استاد در کل تدریس کرده با COUNT و GROUP BY محاسبه می‌شود.

- کوئری ۱۴ - تعداد ثبت‌نام در هر ترم:

```
SELECT sem.TermInfo, COUNT(e.EnrollmentID) AS تعداد_ثبت_نام  
FROM Enrollment e  
JOIN Section sec ON e.SectionID = sec.SectionID  
JOIN Semester sem ON sec.SemesterID = sem.SemesterID  
GROUP BY sem.TermInfo  
ORDER BY sem.TermInfo;
```

این کوئری نشان می‌دهد که در هر ترم تحصیلی چه تعداد ثبت‌نام انجام شده است و نتایج بر اساس ترم مرتب شده‌اند.

- کوئری ۱۵ - دانشجویان با نمره بالای ۱۵:

```
SELECT s.FirstName, s.LastName, c.CourseName, e.Grade  
FROM Enrollment e  
JOIN Students s ON e.StudentID = s.StudentID
```

```
JOIN Section sec ON e.SectionID = sec.SectionID
```

```
JOIN Courses c ON sec.CourseID = c.CourseID
```

```
WHERE e.Grade > 15;
```

با ترکیب JOIN و شرط WHERE ، دانشجویانی که در هر درسی نمره بالای ۱۵ کسب کرده‌اند به همراه نام درس و نمره نمایش داده می‌شوند.

▪ کوئری ۱۶- دانشجویانی با بیشترین واحد ثبت‌نام‌شده:

```
SELECT s.FirstName, s.LastName, SUM(c.Credits) AS مجموع_واحد
```

```
FROM Enrollment e
```

```
JOIN Students s ON e.StudentID = s.StudentID
```

```
JOIN Section sec ON e.SectionID = sec.SectionID
```

```
JOIN Courses c ON sec.CourseID = c.CourseID
```

```
WHERE e.Status = 'Registered'
```

```
GROUP BY s.StudentID
```

```
ORDER BY مجموع_واحد DESC
```

```
LIMIT 5;
```

این کوئری با استفاده از SUM مجموع واحدهای هر دانشجو را حساب کرده، آن‌ها را به صورت نزولی مرتب می‌کند و ۵ دانشجوی با بیشترین واحد را نمایش می‌دهد.

▪ کوئری ۱۷- دانشجویانی که هیچ درسی انتخاب نکرده‌اند:

```
SELECT s.FirstName, s.LastName
```

```
FROM Students s
```

```
LEFT JOIN Enrollment e ON s.StudentID = e.StudentID  
WHERE e.EnrollmentID IS NULL;
```

با استفاده از LEFT JOIN و بررسی NULL بودن EnrollmentID ، دانشجویانی که هیچ ثبت‌نامی ندارند شناسایی می‌شوند.

▪ کوئری ۱۸- کلاس‌هایی که ظرفیتشان پر شده:

```
SELECT c.CourseName, sec.Capacity, COUNT(e.EnrollmentID) AS ثبت_نام_شده  
FROM Section sec  
JOIN Courses c ON sec.CourseID = c.CourseID  
LEFT JOIN Enrollment e ON sec.SectionID = e.SectionID AND e.Status = 'Registered'  
GROUP BY sec.SectionID  
HAVING COUNT(e.EnrollmentID) >= sec.Capacity;
```

این کوئری با استفاده از HAVING (که برای فیلتر کردن نتایج بعد از GROUP BY به کار می‌رود)، کلاس‌هایی را که تعداد ثبت‌نام‌های تأییدشده به ظرفیت رسیده یا از آن بیشتر شده نمایش می‌دهد.

۷. نتیجه گیری

در این پروژه، یک پایگاه داده رابطه‌ای کامل برای سیستم انتخاب واحد دانشگاهی طراحی و پیاده‌سازی شد. طراحی این سیستم با رسم نمودار ER آغاز شد و پس از شناسایی موجودیت‌ها و روابط میان آن‌ها، ۸ جدول اصلی در محیط Navicat ایجاد و کلیدهای اصلی و خارجی آن‌ها تعریف شدند. سپس داده‌های آزمایشی واقع‌گرایانه‌ای درج شد و ۱۸ کوئری SQL در سه سطح ساده، JOIN و تجمیعی نوشته شد تا صحت طراحی پایگاه داده و کارایی ساختار آن به اثبات برسد.

این پروژه نشان داد که با نرمال‌سازی صحیح جداول و تعریف دقیق روابط، می‌توان اطلاعات پیچیده یک سیستم دانشگاهی را به صورت کارآمد مدیریت کرد و پرسش‌های مختلف را با کوئری‌های بهینه پاسخ داد.